

FINAL

TÉCNICAS DE COMPILACIÓN

Profesor: Francisco Ameri Lopez Lozano

Alumno:

- German Rodrigo Arreguez

Resumen

El objetivo de este Trabajo Final es extender las funcionalidades del programa realizado como Trabajo Práctico Nro. 2. El programa a desarrollar tiene como objetivo tomar un archivo de código fuente en C, generar como salida una verificación gramatical reportando errores en caso de existir, generar código intermedio y realizar alguna optimización al código intermedio.

Consigna

El objetivo de este Trabajo Final es mejorar el filtro generado en el Trabajo Práctico Nro. 2. Dado un archivo de entrada en C, se debe generar como salida el reporte de errores en caso de existir. Para lograr esto se debe construir un parser que tenga como mínimo la implementación de los siguientes puntos:

- Reconocimiento de bloques de código delimitados por llaves y controlar balance de apertura y cierre.
- Verificación de la estructura de las operaciones aritmético/lógicas y las variables o números afectadas.
- Verificación de la correcta utilización del punto y coma para la terminación de instrucciones. Balance de llaves, corchetes y paréntesis.
- Tabla de símbolos.
- Llamado a funciones de usuario.

Si la fase de verificación gramatical no ha encontrado errores, se debe proceder a: 1.

Detectar variables y funciones declaradas pero no utilizadas y viceversa. 2. Generar la versión en código intermedio utilizando código de tres direcciones, el cual fue abordado en clases y se encuentra explicado con mayor profundidad en la bibliografía de la materia.

3. Realizar optimizaciones simples como propagación de constantes y eliminación de operaciones repetidas.

En resumen, dado un código fuente de entrada el programa deberá generar los siguientes archivos de salida:

1. La tabla de símbolos para todos los contextos.
2. La versión en código de tres direcciones del código fuente.
3. La versión optimizada del código de tres direcciones.

Solución Propuesta

Mejora del Analizador Gramatical

El archivo de gramática de ANTLR fue actualizado para incorporar nuevas reglas esenciales que faciliten la detección de errores y la creación de la Tabla de Símbolos.

Desarrollo del Visitor

Se desarrolló un visitor utilizando ANTLR para generar código intermedio en tres direcciones (C3D). Este código sirve como un puente entre el código de alto nivel y el lenguaje máquina, manteniendo la lógica esencial y facilitando su optimización o traducción.

El visitor se basa en un patrón de visitador para recorrer el árbol de sintaxis abstracta (AST), procesando nodos según las reglas definidas en la gramática. Se implementan funcionalidades para gestionar variables temporales, etiquetas, y estructuras de control como bucles y condicionales. Además, incluye mecanismos para manejar llamadas a funciones, retornos y validaciones, garantizando que el código intermedio sea consistente y funcional.

Generación de temporales y etiquetas

- `generadorNombresTemporales()`: Genera nombres únicos para variables temporales en formato `t0`, `t1`, etc. Estas variables almacenan resultados intermedios durante la generación de código.
- `nuevoLabel()`: Genera nombres únicos para etiquetas en formato `l0`, `l1`, etc., utilizadas en estructuras de control como bucles y condicionales.

Procesamiento de nodos del árbol de sintaxis

- `visit(ParseTree tree)`: Inicia la visita al árbol sintáctico desde un nodo específico,

procesando sus hijos y generando el código correspondiente.

- visitAllHijos(RuleContext ctx): Recorre y procesa todos los hijos de un nodo de contexto, manejando la indentación para reflejar la jerarquía del árbol.

Visitadores específicos

- visitInstrucciones(InstruccionesContext ctx): Procesa nodos que contienen listas de instrucciones.
- visitDeclaracion(DeclaracionContext ctx): Genera código para declaraciones de variables, gestionando asignaciones si están presentes.

3



- visitAsignacion(AsignacionContext ctx): Genera código para asignaciones como $x = 5$.
- visitBucleFor(BucleForContext ctx): Genera el código intermedio necesario para la ejecución de bucles for.
- visitBucleWhile(BucleWhileContext ctx): Genera código para bucles while, manejando etiquetas de inicio y fin.
- visitCondif(CondifContext ctx): Procesa estructuras condicionales (if y else) con etiquetas para bifurcaciones y saltos.
- visitDefinicionFuncion(DefinicionFuncionContext ctx): Genera código para la definición de funciones, asignando etiquetas únicas para identificarlas.
- visitLlamadaFuncion(LlamadaFuncionContext ctx): Maneja llamadas a funciones, asignando y utilizando etiquetas específicas para gestionar el flujo.

Manejo del código generado

- imprimirCodigo(): Extrae y organiza elementos desde las pilas temporales para generar líneas de código intermedio.
- imprimirCodigoBucles(): Similar a imprimirCodigo(), pero ajustado para incrementar contadores en bucles.

Errores y salidas

- errores: Lista que almacena nodos con errores detectados durante la visita al árbol.
- out y outOptimizado: Escritores para guardar el código intermedio y su versión optimizada en archivos de texto.

Conclusión

Desarrollar un compilador fue un reto que nos llevó a meternos de lleno en la teoría de lenguajes y en cómo funcionan los analizadores. Uno de los problemas más complicados fue crear un algoritmo de análisis sintáctico que fuera eficiente y se adaptara a nuestra gramática, especialmente cuando había ambigüedades o recursividad. Después de investigar y probar distintas ideas, logramos construir un visitor que validará correctamente el código fuente.

Además, optimizar el código intermedio fue un desafío súper interesante. Aplicando técnicas

como la propagación de constantes y eliminando expresiones redundantes, conseguimos generar un código mucho más limpio y eficiente, lo que nos dio una visión más práctica de cómo mejorar el rendimiento en proyectos de este tipo.